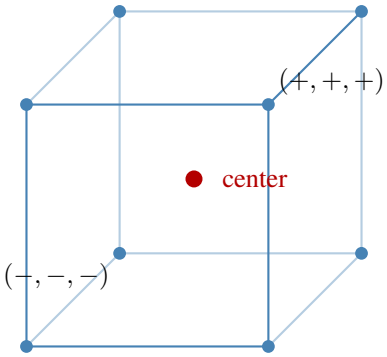
Design of Experiments

A Practical Guide

From Theory to Automated Analysis



First Edition

2025

Design of Experiments: A Practical Guide

First Edition, 2025

This work is released under the MIT License.
You are free to copy, modify, and distribute this work.

Typeset in TEX Gyre Termes.
Compiled with LuaLATEX.

Contents

Preface **7**

I Foundations **9**

1 Why Design of Experiments? **11**
1.1 The Cost of Unplanned Experiments 11
1.2 The Three Principles of DOE 12
1.2.1 Randomization 12
1.2.2 Replication 13
1.2.3 Blocking 13
1.3 The OVAT Trap: A Quantitative Example 14
1.3.1 OVAT Analysis 14
1.3.2 Factorial Analysis 14
1.4 A Concrete Example: Web Server Tuning 15
1.4.1 OVAT Gets the Wrong Answer 15
1.4.2 DOE Gets the Right Answer 15
1.5 DOE in Everyday Life 16

2 Statistical Foundations **19**
2.1 The Linear Model 19
2.2 Coded Variables 20
2.3 Estimating Effects 20
2.3.1 Main Effects 20
2.3.2 Interaction Effects 20
2.3.3 Percentage Contribution 21
2.4 Confidence Intervals 21
2.5 Analysis of Variance (ANOVA) 21
2.6 The R^2 Statistic 22

II Design Types 23

3 Full Factorial Designs 25
3.1 The 2^k Factorial . . . 25
3.1.1 Run Count . . . 25
3.1.2 The Design Matrix . . . 25
3.1.3 Geometric Interpretation . . . 26
3.2 Mixed-Level Factorials . . . 27
3.3 When to Use Full Factorials . . . 27

4 Fractional Factorial Designs 29
4.1 The Problem of Exponential Growth . . . 29
4.2 The 2^{k-p} Design . . . 29
4.3 Resolution . . . 30
4.4 Generator Strings and Aliasing . . . 30
4.5 Practical Guidance . . . 30

5 Screening Designs 33
5.1 Plackett–Burman Designs . . . 33
5.1.1 Properties . . . 33
5.1.2 The Screening-to-Optimization Pipeline . . . 33

6 Response Surface Designs 35
6.1 Beyond Main Effects . . . 35
6.2 Central Composite Design (CCD) . . . 35
6.2.1 Run Count . . . 36
6.3 Box–Behnken Design . . . 37
6.3.1 When to Use Box–Behnken vs. CCD . . . 38
6.4 Latin Hypercube Sampling . . . 38
6.4.1 Construction . . . 38
6.4.2 When to Use LHS . . . 38
6.5 Quick Reference: Choosing a Design . . . 38

III Analysis and Optimization 41

7 Analyzing Results 43
7.1 The Pareto Chart . . . 43
7.1.1 Reading a Pareto Chart . . . 43
7.2 Main Effects Plots . . . 44
7.3 Interaction Plots . . . 44

7.4 Reading Tool Output 44
7.5 Complete Worked Example 45
7.5.1 Step 1: The Raw Data 45
7.5.2 Step 2: Calculate the Main Effect of A (Cache) . . 46
7.5.3 Step 3: Calculate Main Effects of B and C 46
7.5.4 Step 4: Calculate the AB Interaction 46
7.5.5 Step 5: Pareto Ranking 47
7.5.6 Step 6: Confidence Intervals 47
7.5.7 Step 7: Conclusions 47
7.6 Statistical Significance 48
7.6.1 Using Confidence Intervals 48
7.6.2 Practical vs. Statistical Significance 48

8 Response Surface Methodology 49
8.1 From Effects to Models 49
8.2 What Is a Response Surface? 49
8.3 Fitting the Model 50
8.4 Model Adequacy 50
8.4.1 R^2 and Adjusted R^2 50
8.4.2 Residual Analysis 50
8.5 Finding the Optimum 51
8.6 When the Optimum Is Outside Your Design Space 51
8.7 Multi-Response Optimization 52

IV Practice 53

9 Planning Your First Experiment 55
9.1 The Pre-Experiment Checklist 55
9.1.1 1. What Am I Trying to Learn or Optimize? 55
9.1.2 2. What Factors Can I Control? 56
9.1.3 3. What Are Reasonable Low and High Levels? . . 56
9.1.4 4. What Will I Measure? 56
9.1.5 5. How Noisy Is My System? 57
9.1.6 6. How Many Runs Can I Afford? 57
9.1.7 7. Are There Nuisance Variables I Should Block On? 57
9.2 Worked Example: Planning a Web Server Optimization . . 57
9.3 Common Pitfalls in Planning 58

10 The Sequential Experimentation Strategy **61**
10.1 Phase 1: Screening 61
10.2 Phase 2: Characterization 61
10.3 Phase 3: Optimization 62
10.4 Phase 4: Confirmation 62

11 Applications **63**
11.1 Software Performance Benchmarking 63
11.1.1 Example: Database Configuration 63
11.1.2 Example: Compiler Flags 64
11.2 Scientific and Engineering Experiments 64
11.2.1 Example: Chemical Reactor Optimization 64
11.2.2 Example: Materials Science 64
11.2.3 Example: Microservice Configuration 65
11.3 Machine Learning Hyperparameter Tuning 65

12 Common Mistakes and How to Avoid Them **67**
12.1 Choosing Factor Levels Too Close Together 67
12.2 Not Randomizing Run Order 67
12.3 Changing Other Things Between Runs 68
12.4 Testing Too Many Factors Without Screening 68
12.5 Ignoring Interactions 68
12.6 Not Running Confirmation Experiments 68
12.7 Confusing Replication with Repetition 69
12.8 Over-Engineering the First Experiment 69

13 Using the DOE Helper Tool **71**
13.1 Installation 71
13.2 Configuration File 71
13.3 Configuration Reference 72
13.3.1 Metadata Fields 72
13.3.2 Factor Fields 72
13.3.3 Response Fields 73
13.3.4 Runner Fields 74
13.3.5 Settings Fields 74
13.3.6 Fixed Factors 75
13.4 Supported Operations 76
13.5 Workflow 76
13.6 Test Script Protocol 76
13.7 Analysis Output 77

13.8 Troubleshooting 77
13.8.1 “My Effects Are All Near Zero” 77
13.8.2 “R^2 Is Very Low” 78
13.8.3 “Confidence Intervals Are Huge” 78
13.8.4 “The Predicted Optimum Is Unrealistic” 78
13.8.5 “Runner Script Fails on Some Runs” 79

A Mathematical Reference **81**
A.1 Effect Estimation Formulas 81
A.2 Response Surface Model 82
A.3 Minimum Run Counts by Design 82

B Glossary **83**

References **85**

Preface

This book teaches the theory and practice of Design of Experiments (DOE)—a systematic method for planning experiments that extracts the maximum information from the minimum number of runs. It is written for engineers, scientists, and anyone who runs experiments, benchmarks, or tests and wants to do so more efficiently.

DOE is not new. Ronald Fisher developed the foundations in the 1920s and 1930s for agricultural experiments at Rothamsted. George Box, Genichi Taguchi, and others extended it to manufacturing, quality engineering, and process optimization. Yet the technique remains underused, especially in software engineering and technology, where practitioners often default to changing one variable at a time or running exhaustive grid searches.

This guide takes a practical approach. Each chapter introduces the theory you need, then shows you how to apply it. Mathematical derivations are included where they build intuition, but the emphasis is on understanding *when* and *why* to use each technique, not on proofs.

The examples throughout this book can be run using the open-source DOE Helper Tool that accompanies it. The tool reads a JSON configuration file, generates an optimized experimental design, produces an executable runner script, and analyzes the results with statistical summaries, plots, and reports.

Who this book is for:

- Engineers optimizing systems, processes, or software configurations
- Scientists planning laboratory or field experiments
- Data scientists evaluating model hyperparameters
- Quality engineers improving manufacturing processes
- Anyone who has ever wondered: "Which of these 10 knobs actually matters?"

Prerequisites: Basic familiarity with mean, variance, and standard deviation. No prior statistics coursework is assumed.

Part I

Foundations

Chapter 1

Why Design of Experiments?

1.1 The Cost of Unplanned Experiments

Every experiment costs something—time, money, materials, compute cycles. A poorly planned experiment wastes these resources and may lead to wrong conclusions. A well-designed experiment extracts the maximum learning from each run.

The Cake Analogy

Imagine you are baking a cake and want to make it taste better. You could try changing the sugar amount, then the oven temperature, then the baking time—one thing at a time. After dozens of attempts, you might find a decent recipe. But you would never discover that a *specific combination* of less sugar *with* higher temperature produces the best result, because you never tested them together. DOE solves exactly this problem.

Consider a team tuning a database. They have five configuration knobs they suspect affect performance: buffer pool size, I/O capacity, connection limit, query cache, and thread pool size. The one-variable-at-a-time (OVAT) approach would be:

1. Fix all knobs at their defaults.
2. Change buffer pool size. Measure.
3. Reset buffer pool. Change I/O capacity. Measure.
4. Continue for each knob.

This requires at least 10 runs (2 per knob) and only measures each knob's

effect *in isolation, at the default settings of every other knob*. It tells you nothing about how the knobs interact.

A Plackett–Burman screening design tests all five knobs in just 8 runs and estimates every main effect simultaneously. A full factorial on the two most important knobs then takes 4 more runs and reveals their interaction. Twelve runs total, and you know more than OVAT would teach you in fifty.

1.2 The Three Principles of DOE

Fisher established three principles that remain the foundation of every well-designed experiment:

1.2.1 Randomization

Randomization

Randomly assign the order in which experimental runs are performed to protect against unknown lurking variables.

If you run all low-temperature experiments in the morning and all high-temperature experiments in the afternoon, any morning-to-afternoon drift (ambient temperature, operator alertness, equipment warm-up) becomes confounded with your temperature factor. You cannot tell whether the observed difference is due to temperature or time of day.

Randomization does not eliminate lurking variables—it distributes their effects equally across all treatments, on average. This ensures that the statistical tests remain valid.

Mathematically, if Y_{ij} is the response for factor level i in run order j, and there is a time trend δ_j, then without randomization:

$$E[Y_{ij}] = \mu + \tau_i + \delta_j$$

where τ_i and δ_j may be confounded. With randomization, δ_j is equally likely to affect any treatment, so:

$$E[\bar{Y}_{i\cdot}] = \mu + \tau_i$$

and the time trend averages out.

1.2.2 Replication

Replication

Repeat experimental runs to estimate experimental error and increase the precision of effect estimates.

A single run at each factor combination tells you the response at that combination, but it does not tell you how much that response varies. Without an estimate of variability, you cannot determine whether an observed difference is real or just noise.

Replication provides:

- An estimate of the experimental error variance σ^2
- More precise estimates of effects (standard error decreases as $1/\sqrt{n}$)
- The ability to construct confidence intervals and perform significance tests

Replication vs. Repetition

Replication means independently setting up and running the experiment again. **Repetition** means measuring the same setup multiple times. Repetition estimates measurement error, but replication estimates the full experimental error, including setup variation. DOE requires replication.

1.2.3 Blocking

Blocking

Group experimental runs into blocks of similar conditions to remove known sources of variation from the experimental error.

If you must run experiments over two days, make each day a block. Within each block (day), randomize the run order. The day-to-day variation is then accounted for in the analysis and does not inflate the error estimate.

The model becomes:

$$Y_{ijk} = \mu + \tau_i + \beta_j + \epsilon_{ijk}$$

where β_j is the block effect. Because blocks are orthogonal to treatments in a balanced design, the treatment effects τ_i are estimated without bias.

1.3 The OVAT Trap: A Quantitative Example

Suppose the true relationship between yield and two factors (temperature T and pressure P) is:

$$Y = 50 + 10T + 5P + 8TP + \epsilon$$

where $T, P \in \{-1, +1\}$ (coded levels) and $\epsilon \sim N(0, 1)$.

1.3.1 OVAT Analysis

1. Run at $T = -1, P = -1$: $Y = 50 - 10 - 5 + 8 = 43$
2. Change T to $+1$: $Y = 50 + 10 - 5 - 8 = 47$
3. Conclude: Temperature effect $= 47 - 43 = 4$

But the true main effect of temperature is $+10$, not $+4$! The OVAT estimate is biased because it was measured at $P = -1$, where the interaction TP reduces temperature's apparent effect. At $P = +1$, the temperature effect would appear as $+18$.

1.3.2 Factorial Analysis

A 2^2 factorial runs all four combinations:

Run	T	P	Y (expected)
1	-1	-1	43
2	$+1$	-1	47
3	-1	$+1$	37
4	$+1$	$+1$	73

Main effect of T:

$$\text{ME}_T = \frac{(Y_2 + Y_4)}{2} - \frac{(Y_1 + Y_3)}{2} = \frac{47 + 73}{2} - \frac{43 + 37}{2} = 60 - 40 = 20$$

Wait—the formula gives 20, but we said the coefficient was 10. In the effects model, the main effect is *twice* the coefficient because we move from -1 to $+1$ (a change of 2 units). So $\text{ME}_T/2 = 10$, exactly the true coefficient.

The interaction effect:

$$\text{IE}_{TP} = \frac{(Y_1 + Y_4)}{2} - \frac{(Y_2 + Y_3)}{2} = \frac{43 + 73}{2} - \frac{47 + 37}{2} = 58 - 42 = 16$$

Again, $\text{IE}_{TP}/2 = 8$, the true interaction coefficient. The factorial design correctly estimates both effects and their interaction, using the same number of runs where each piece of data contributes to every estimate.

Hidden Replication in Factorials

In a 2^k factorial, every effect estimate uses *all* 2^k observations. Each observation plays a role in estimating every effect. This is why factorials are so efficient—there is no wasted data.

1.4 A Concrete Example: Web Server Tuning

This example makes the OVAT trap viscerally clear. Suppose you are optimizing a web server and suspect two things matter: **thread count** (10 or 100) and **cache size** (64 MB or 512 MB). The true relationship (which you do not know yet) is:

Thread Count	Cache Size	Throughput (req/sec)
10	64 MB	500
100	64 MB	600
10	512 MB	700
100	512 MB	1400

1.4.1 OVAT Gets the Wrong Answer

Starting at (10 threads, 64 MB cache):

1. Change threads to 100, keep cache at 64 MB → throughput goes from 500 to 600. *Conclusion: threads help a little (+100).*
2. Reset threads to 10. Change cache to 512 MB → throughput goes from 500 to 700. *Conclusion: cache helps more (+200).*
3. Final recommendation: use 512 MB cache. Predicted throughput: 700.

1.4.2 DOE Gets the Right Answer

A 2^2 full factorial tests all four combinations:

- Main effect of threads: $\frac{600+1400}{2} - \frac{500+700}{2} = 1000 - 600 = +\mathbf{400}$
- Main effect of cache: $\frac{700+1400}{2} - \frac{500+600}{2} = 1050 - 550 = +\mathbf{500}$
- Interaction effect: $+\mathbf{500}$ (threads and cache amplify each other)

- Recommendation: use *both* 100 threads and 512 MB cache. Throughput: **1400**.

Same number of experiments. Completely different—and correct—conclusion. The OVAT approach missed the interaction entirely and left 700 req/sec of throughput on the table.

Interactions Are the Rule, Not the Exception

In the web server example, the cache is only truly effective when there are enough threads to use it. This kind of synergy is extremely common in real systems—database buffer pools that only help with high I/O capacity, compiler optimizations that only matter with specific architecture targets, or machine learning learning rates that only work with certain batch sizes.

1.5 DOE in Everyday Life

The logic of DOE is not confined to laboratories or data centers. You use it—or fail to use it—whenever you try to improve anything with multiple knobs to turn.

Cooking: Testing Seasoning Combinations

Suppose you are perfecting a pasta sauce and want to know how much **salt**, **garlic**, and **chili flake** to add. You could test each seasoning one at a time, holding the others constant. But what if garlic only shines when paired with enough salt to bring out its flavor? A factorial approach—testing all combinations of low/high for each seasoning—lets you discover that the magic is in the *combination* of moderate salt with generous garlic, while chili flake barely matters. Eight small batches tell you more than twenty one-at-a-time tastings.

Gardening: Fertilizer, Watering, and Sunlight

A gardener wants bigger tomatoes. The controllable factors are **fertilizer** (low vs. high), **watering frequency** (daily vs. every other day), and **sunlight exposure** (partial shade vs. full sun). Instead of changing one factor per growing season (which would take three seasons), the gardener plants eight small plots—one for each combination—in

the same season. The result: fertilizer has a huge effect, but *only* when combined with daily watering. Full sun helps regardless. Two weeks of harvest data replace three years of guesswork.

Shopping: Comparing Products Across Multiple Criteria

When buying a laptop, you care about **price**, **battery life**, and **weight**. Sorting a comparison table by just one criterion (say, price) hides the trade-offs. A DOE mindset asks: "Which criterion has the biggest effect on my satisfaction?" and "Do any criteria interact?" (e.g., you only care about weight if battery life is long enough to leave the charger at home). Thinking in terms of factors, levels, and interactions helps you make multi-criteria decisions systematically, even outside a formal experiment.

Chapter 2

Statistical Foundations

2.1 The Linear Model

Before we write down any math, let us understand what we are doing in plain English. We are trying to build a simple equation that predicts our response from the factor settings. Think of it as:

Result = (baseline) + (contribution from factor A) + (contribution from factor B) + (how A and B work together) + (random noise)

Each factor either pushes the result up or pulls it down by some amount, and the interaction term captures any synergy or conflict between factors. The math below just makes this precise.

At the heart of DOE analysis is the general linear model. For a single response Y with k factors:

$$Y = \beta_0 + \sum_{i=1}^{k} \beta_i x_i + \sum_{i<j} \beta_{ij} x_i x_j + \epsilon$$

where:

- β_0 is the grand mean (intercept)
- β_i are the main effect coefficients
- β_{ij} are the interaction coefficients
- $x_i \in \{-1, +1\}$ are the coded factor levels
- $\epsilon \sim N(0, \sigma^2)$ is the random error

For a quadratic (response surface) model, add squared terms:

$$Y = \beta_0 + \sum_{i=1}^{k} \beta_i x_i + \sum_{i<j} \beta_{ij} x_i x_j + \sum_{i=1}^{k} \beta_{ii} x_i^2 + \epsilon$$

2.2 Coded Variables

Coding is just putting all factors on the same scale. If one factor is measured in degrees (150–200) and another in bars (2–6), their raw numbers are not comparable. A coefficient of 5 for temperature and 5 for pressure in raw units would *not* mean they have the same importance, because a 50-degree change is very different from a 4-bar change. Coding transforms both factors to a -1 to $+1$ scale, so a coefficient of 5 for temperature and 5 for pressure genuinely means they have the same effect on the response.

Factor levels are coded to simplify calculations and make effects directly comparable. For a continuous factor with low value L and high value H:

$$x = \frac{\text{actual value} - \frac{L+H}{2}}{\frac{H-L}{2}}$$

This maps $L \to -1$ and $H \to +1$. The center is at $x = 0$. All factors are on the same scale, so their coefficients are directly comparable—a coefficient of 10 means the same magnitude of effect regardless of whether the factor is measured in degrees or bars.

2.3 Estimating Effects

2.3.1 Main Effects

The main effect of factor A is the difference in the average response when A is at its high level versus its low level:

$$\text{ME}_A = \bar{Y}_{A+} - \bar{Y}_{A-}$$

In a 2^k design, each average uses half the total runs, so the standard error of a main effect is:

$$\text{SE}(\text{ME}_A) = \sqrt{\frac{4s^2}{N}} = \frac{2s}{\sqrt{N}}$$

where s^2 is the pooled variance estimate and N is the total number of runs.

2.3.2 Interaction Effects

The two-factor interaction AB measures whether the effect of A depends on the level of B. Compute it as:

$$\text{IE}_{AB} = \frac{1}{2}\big[(\text{effect of } A \text{ at } B_+) - (\text{effect of } A \text{ at } B_-)\big]$$

Equivalently, in a 2^k design:

$$\text{IE}_{AB} = \bar{Y}_{\text{concordant}} - \bar{Y}_{\text{discordant}}$$

where "concordant" runs have A and B both high or both low, and "discordant" runs have one high and one low.

2.3.3 Percentage Contribution

To rank factors by importance, compute each factor's percentage contribution:

$$\text{PC}_i = \frac{|\text{ME}_i|}{\sum_{j=1}^{k} |\text{ME}_j|} \times 100\%$$

This appears on the Pareto chart. The "vital few" factors typically account for 70–80% of the total effect magnitude.

2.4 Confidence Intervals

A 95% confidence interval for a main effect is:

$$\text{ME}_A \pm t_{\alpha/2,\nu} \cdot \text{SE}(\text{ME}_A)$$

where $t_{\alpha/2,\nu}$ is the critical value from the t-distribution with ν degrees of freedom. If the interval contains zero, the effect is not statistically significant at the 5% level.

For a two-level design with n_- and n_+ observations at the low and high levels respectively:

$$\text{SE}(\text{ME}_A) = s_p \sqrt{\frac{1}{n_-} + \frac{1}{n_+}}$$

where s_p is the pooled standard deviation:

$$s_p = \sqrt{\frac{(n_- - 1)s_-^2 + (n_+ - 1)s_+^2}{n_- + n_+ - 2}}$$

2.5 Analysis of Variance (ANOVA)

ANOVA answers a simple question: "How much of the total variation in my results is caused by each factor, and how much is just noise?" It splits the

total variation into pieces—like splitting a pie chart—so you can see which factors account for the biggest slices. If temperature accounts for 60% of the variation, pressure for 15%, and the remaining 25% is unexplained noise, you know exactly where to focus your optimization effort.

ANOVA decomposes the total variation in the response into components attributable to each factor, their interactions, and the residual error:

$$\text{SS}_{\text{Total}} = \sum_i \text{SS}_{A_i} + \sum_{i<j} \text{SS}_{A_i A_j} + \text{SS}_{\text{Error}}$$

For a main effect in a 2^k design:

$$\text{SS}_A = \frac{N}{4} \cdot \text{ME}_A^2$$

The F-statistic for testing whether factor A has a significant effect is:

$$F_A = \frac{\text{SS}_A/1}{\text{SS}_{\text{Error}}/\nu_E} = \frac{\text{MS}_A}{\text{MS}_E}$$

If $F_A > F_{\alpha,1,\nu_E}$, reject the null hypothesis that factor A has no effect.

2.6 The R^2 Statistic

R^2 measures the proportion of variance in the response explained by the model:

$$R^2 = 1 - \frac{\text{SS}_{\text{Error}}}{\text{SS}_{\text{Total}}}$$

The adjusted R^2 penalizes for model complexity:

$$R^2_{\text{adj}} = 1 - \frac{\text{SS}_{\text{Error}}/\nu_E}{\text{SS}_{\text{Total}}/\nu_T}$$

An R^2 near 1.0 means the model explains nearly all the variation. An R^2 below 0.7 suggests important factors or relationships are missing from the model.

Part II

Design Types

Chapter 3

Full Factorial Designs

3.1 The 2^k Factorial

The 2^k full factorial design tests every combination of k factors at 2 levels each. It is the most complete two-level design: it estimates all main effects, all two-factor interactions, and all higher-order interactions with no confounding.

3.1.1 Run Count

Factors (k)	Runs (2^k)	Notes
2	4	Minimum useful design
3	8	Very practical
4	16	Still manageable
5	32	Getting expensive
6	64	Consider fractional
7	128	Fractional almost always better

3.1.2 The Design Matrix

For a 2^3 design with factors A, B, C:

Run	A	B	C	AB	AC	BC	ABC
1	$-$	$-$	$-$	$+$	$+$	$+$	$-$
2	$+$	$-$	$-$	$-$	$-$	$+$	$+$
3	$-$	$+$	$-$	$-$	$+$	$-$	$+$
4	$+$	$+$	$-$	$+$	$-$	$-$	$-$
5	$-$	$-$	$+$	$+$	$-$	$-$	$+$
6	$+$	$-$	$+$	$-$	$+$	$-$	$-$
7	$-$	$+$	$+$	$-$	$-$	$+$	$-$
8	$+$	$+$	$+$	$+$	$+$	$+$	$+$

The interaction columns are computed by multiplying the corresponding main effect columns element-wise. This orthogonality property—every column has an equal number of $+$ and $-$ signs, and the inner product of any two columns is zero—is what makes the effect estimates independent and unbiased.

3.1.3 Geometric Interpretation

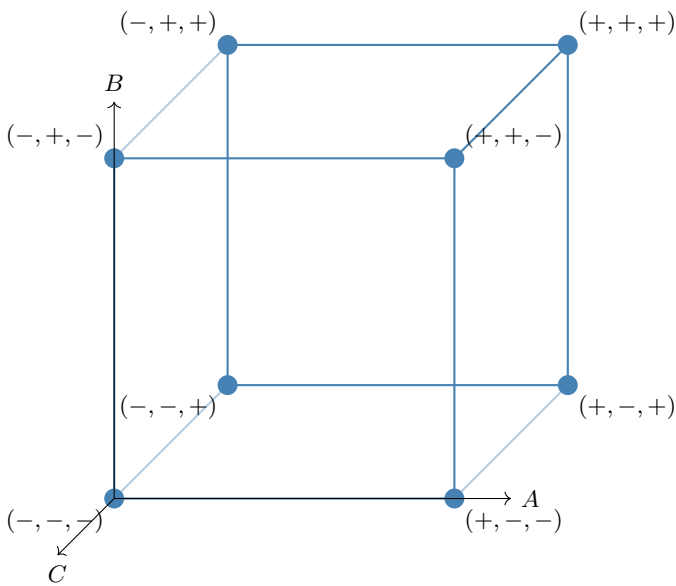


Figure 3.1: A 2^3 full factorial as a cube. Each vertex is an experimental run. The main effect of A is the average of the four right-face responses minus the average of the four left-face responses.

3.2 Mixed-Level Factorials

Full factorials are not limited to two levels. A $3 \times 2 \times 4$ design tests one factor at 3 levels, one at 2, and one at 4, yielding $3 \times 2 \times 4 = 24$ runs. The analysis uses the range of level means (maximum mean minus minimum mean) instead of the simple high-minus-low formula.

3.3 When to Use Full Factorials

- 2–4 factors: almost always the right choice
- You need to estimate all interactions, not just main effects
- Runs are cheap relative to the information gained
- You suspect important interactions exist

Chapter 4

Fractional Factorial Designs

4.1 The Problem of Exponential Growth

As the number of factors increases, full factorials become impractical. With 7 factors at 2 levels, a full factorial requires $2^7 = 128$ runs. But in practice, the *effect sparsity principle* states that most of the variation is explained by main effects and low-order interactions. Higher-order interactions (ABC, $ABCD$, etc.) are usually negligible.

Fractional factorials exploit this by deliberately confounding higher-order interactions with main effects or lower-order interactions, trading information about unlikely-to-matter effects for a dramatic reduction in runs.

4.2 The 2^{k-p} Design

A 2^{k-p} fractional factorial uses 2^{k-p} runs to study k factors. The integer p is the *degree of fractionation*: each increase in p halves the number of runs.

Design	Factors	Runs	Fraction
2^{4-1}	4	8	$\frac{1}{2}$
2^{5-1}	5	16	$\frac{1}{2}$
2^{5-2}	5	8	$\frac{1}{4}$
2^{7-4}	7	8	$\frac{1}{16}$

4.3 Resolution

The *resolution* of a fractional factorial describes the severity of the confounding:

Resolution	Confounding Pattern
III	Main effects aliased with 2-factor interactions
IV	Main effects clear; 2FIs aliased with other 2FIs
V	Main effects and 2FIs clear; 2FIs aliased with 3FIs

Choosing Resolution

Use Resolution III for initial screening (many factors, few runs). Use Resolution IV or V when you suspect important 2-factor interactions and want to estimate them separately from main effects. The higher the resolution, the more runs are required.

4.4 Generator Strings and Aliasing

A fractional factorial is defined by its *generators*—the rules that assign aliased factor columns. For a 2^{5-2}_{III} design, the base factors are A, B, C (requiring $2^3 = 8$ runs), and the two additional factors are defined as:

$$D = AB$$
$$E = AC$$

The *defining relation* is the set of column products equal to I (the identity):

$$I = ABD = ACE = BCDE$$

Any two columns whose product appears in the defining relation are aliased (confounded). For example, $A = BD = CE$ and $B = AD = CDE$.

4.5 Practical Guidance

1. If you have $k \leq 4$ factors, use a full factorial.
2. If $5 \leq k \leq 7$, use a Resolution IV or V fraction.

3. If $k > 7$ and you are screening, use Resolution III or a Plackett–Burman.
4. Always check the alias structure before running the experiment to ensure the effects you care about are not aliased with each other.

Chapter 5

Screening Designs

5.1 Plackett–Burman Designs

When you have many factors (5–20 or more) and your primary goal is to identify the *vital few* that have large main effects, Plackett–Burman (PB) designs are the most efficient choice.

PB designs are 2-level designs where the number of runs is always a multiple of 4. They can screen up to $N - 1$ factors in N runs:

Runs (N)	Max Factors ($N - 1$)
8	7
12	11
16	15
20	19
24	23

5.1.1 Properties

- All main effects are estimated with equal precision.
- Main effects are partially confounded with two-factor interactions (complex aliasing, unlike the clean aliasing of fractional factorials).
- The design is *not* suitable for estimating interactions—it is a pure screening tool.

5.1.2 The Screening-to-Optimization Pipeline

PB designs are almost always the first step in a sequential strategy:

1. **Screen** with PB: Run N experiments, identify the 3–5 factors with the largest main effects.
2. **Characterize** with a full or fractional factorial on the important factors.
3. **Optimize** with a response surface design (CCD or Box–Behnken).

Chapter 6

Response Surface Designs

6.1 Beyond Main Effects

Screening and factorial designs estimate linear effects: how much does the response change when a factor moves from low to high? But many real systems have *curvature*—the optimum is not at an extreme, but somewhere in the middle. To find this optimum, we need a quadratic model:

$$Y = \beta_0 + \sum_i \beta_i x_i + \sum_{i<j} \beta_{ij} x_i x_j + \sum_i \beta_{ii} x_i^2 + \epsilon$$

Response surface designs provide the data points needed to fit this model. They include runs at the center of the factor space (to estimate curvature) and, for CCD, star points beyond the factorial range.

6.2 Central Composite Design (CCD)

The CCD consists of three components:

1. **Factorial points:** A 2^k (or 2^{k-p}) factorial at the corners.
2. **Axial (star) points:** $2k$ points along each axis at distance α from the center.
3. **Center points:** Several runs at the center of the design space.

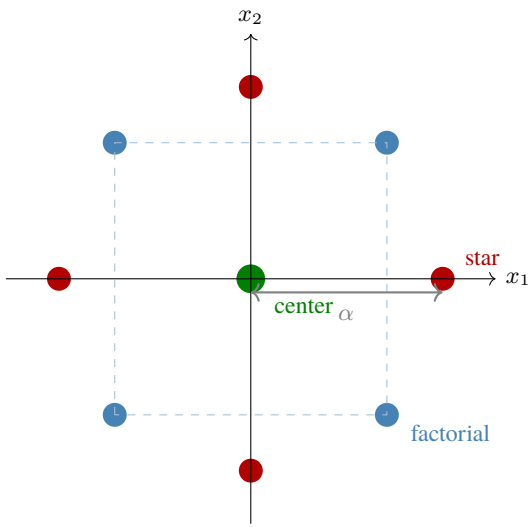


Figure 6.1: Central Composite Design for 2 factors. Blue squares are factorial points, red circles are star (axial) points, and the green circle is the center point.

The value of α determines the design's properties:

- $\alpha = 1$: Face-centered CCD (star points on the face of the cube). Does not require runs outside the factor range, but the quadratic fit is less precise.
- $\alpha = \sqrt{k}$: Spherical CCD. Star points lie on the sphere circumscribing the factorial cube.
- $\alpha = (2^k)^{1/4}$: Rotatable CCD. The prediction variance is constant at all points equidistant from the center, making the model equally precise in all directions.

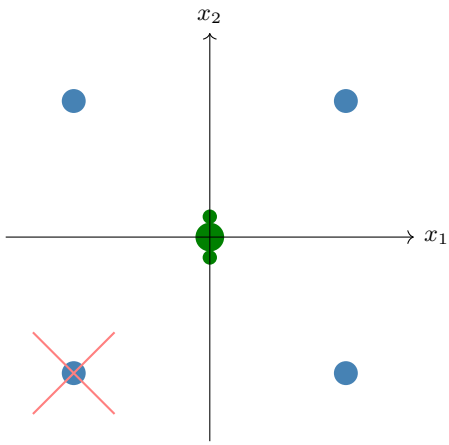
6.2.1 Run Count

Factors	Factorial	Star	Center	Total
2	4	4	5	13
3	8	6	6	20
4	16	8	7	31
5	32	10	8	50

6.3 Box–Behnken Design

Box–Behnken Advantage

Box–Behnken designs avoid extreme corners of the design space. Every run has at most two factors at their extreme levels; the remaining factors are at the center. This makes them safer for physical experiments where corner points are dangerous or infeasible.



(x_3 at center for each pair)

Figure 6.2: Box–Behnken design concept. The extreme corner (crossed out) is not tested. Each pair of factors is tested at all four edge combinations while the third stays at center.

For 3 factors, Box–Behnken requires 15 runs (12 edge-midpoint runs + 3 center replicates), compared to 20 for a CCD. For 4 factors: 27 runs.

6.3.1 When to Use Box–Behnken vs. CCD

Criterion	CCD	Box–Behnken
Avoids extreme corners	No	Yes
Points outside factor range	Yes (star)	No
Minimum factors	2	3
Sequential augmentation from 2^k	Yes	No
Rotatable option	Yes	No
Fewer runs (3–4 factors)	Depends	Often

6.4 Latin Hypercube Sampling

Latin Hypercube Sampling (LHS) is a space-filling design used primarily for computer experiments and simulations where the response is deterministic (no random error).

6.4.1 Construction

For n samples and k factors:

1. Divide each factor's range into n equal intervals.
2. Place exactly one sample point in each interval of each factor.
3. The placement within each interval is random.

This guarantees that the projection onto any single factor axis covers the entire range uniformly—a property called *stratification*.

6.4.2 When to Use LHS

- Computer experiments with deterministic outputs (no noise)
- High-dimensional spaces (many factors) where factorial designs are infeasible
- Building surrogate models (kriging, Gaussian processes)
- Sensitivity analysis and uncertainty quantification

6.5 Quick Reference: Choosing a Design

Design	Best For	Min Runs	Interactions?	Finds C
Full Factorial	Complete picture	2^k	All	
Fractional Fact.	Screening + some 2FIs	2^{k-p}	Some	
Plackett–Burman	Fast screening	$k+1$	No	
Latin Hypercube	Exploration	Your choice	No	
CCD	Finding the optimum	2^k+2k+c	Yes	Y
Box–Behnken	Safe optimization	$\sim$ 15–46	Yes	Y

The Decision Rule

2–4 factors, want everything: Full Factorial. **5+ factors, screening:** Plackett–Burman. **2–5 factors, finding the optimum:** CCD (corners OK) or Box–Behnken (avoid corners). **Computer simulation:** Latin Hypercube.

Part III

Analysis and Optimization

Chapter 7

Analyzing Results

7.1 The Pareto Chart

The Pareto chart is the single most useful visualization in DOE. It displays factors sorted by the absolute magnitude of their main effects, with a cumulative percentage line overlaid.

The 80/20 Rule

The Pareto principle states that roughly 80% of the total effect magnitude is caused by 20% of the factors. The Pareto chart's cumulative line helps you identify these "vital few" factors. Factors below the 80% line are candidates for being fixed at their cheapest or most convenient level.

7.1.1 Reading a Pareto Chart

1. Factors are sorted from largest to smallest absolute effect (top to bottom or left to right).
2. The bar length represents the magnitude of the main effect.
3. The cumulative line shows the running total as a percentage of the sum of all effect magnitudes.
4. The 80% reference line marks where the "vital few" end and the "trivial many" begin.

7.2 Main Effects Plots

A main effects plot shows the mean response at each level of a factor, connected by line segments. The slope of the line indicates the magnitude and direction of the effect.

- A steep line means a large effect.
- A flat line means the factor has little influence.
- A non-linear pattern (for >2 levels) suggests curvature.

7.3 Interaction Plots

An interaction plot shows the mean response for each combination of two factors. If the lines are parallel, there is no interaction. If they cross or diverge, the interaction is significant.

How to Spot Interactions

Parallel lines = no interaction (the effect of A is the same regardless of B's level). **Non-parallel lines** = interaction. **Crossing lines** = strong interaction (the best level of A depends on B's level—this completely changes your recommendation).

7.4 Reading Tool Output

When you run the analysis tool, the output looks like this:

```
=== Main Effects: throughput ===
Factor              Effect   Std Error   % Contribution
-----------------------------------------------------
buffer_pool_size   +850.3       45.2          62.1%
worker_threads     +312.7        45.2          22.8%
gc_heap_size       +142.1        45.2          10.4%
log_level           -64.5        45.2           4.7%
```

How to read each column:

- **Effect:** How much the response changes when the factor goes from low to high. Positive means "high level produces more." Negative means "high level produces less."
- **Std Error:** The uncertainty in the effect estimate. Smaller is more precise. If the effect is much larger than its std error (>2–3×), the effect is likely real, not noise.

- **% Contribution:** How much of the total variation this factor explains. The Pareto chart visualizes this. Factors totaling ∼80% are the "vital few."

Interpreting Confidence Intervals

```
temperature:  effect=+8.5, 95% CI=[+3.2, +13.8]
pressure:     effect=+1.2, 95% CI=[-4.1, +6.5]
```

Temperature: The CI is entirely positive, so the effect is statistically significant. Temperature definitely matters.
Pressure: The CI includes zero, so the effect is NOT statistically significant. The observed $+1.2$ could be noise.

7.5 Complete Worked Example

Let us walk through a full analysis of a 2^3 factorial experiment, step by step, with real numbers. Suppose we are studying the effect of three factors on the response time (in milliseconds) of an API endpoint:

- **A**: Cache enabled (-1 = off, $+1$ = on)
- **B**: Thread pool size (-1 = 4 threads, $+1$ = 16 threads)
- **C**: Compression (-1 = off, $+1$ = on)

7.5.1 Step 1: The Raw Data

Run	A	B	C	AB	Response (ms)
1	-1	-1	-1	$+1$	120
2	$+1$	-1	-1	-1	75
3	-1	$+1$	-1	-1	95
4	$+1$	$+1$	-1	$+1$	40
5	-1	-1	$+1$	$+1$	130
6	$+1$	-1	$+1$	-1	82
7	-1	$+1$	$+1$	-1	105
8	$+1$	$+1$	$+1$	$+1$	48

The grand mean is $\bar{y} = (120+75+95+40+130+82+105+48)/8 = 86.875$ ms.

7.5.2 Step 2: Calculate the Main Effect of A (Cache)

Average response when $A = +1$ (cache on): $\bar{y}_{A+} = (75+40+82+48)/4 = 61.25$

Average response when $A = -1$ (cache off): $\bar{y}_{A-} = (120+95+130+105)/4 = 112.50$

$$\text{ME}_A = \bar{y}_{A+} - \bar{y}_{A-} = 61.25 - 112.50 = -51.25 \text{ ms}$$

The negative sign means cache *reduces* response time (good!). Enabling the cache cuts response time by an average of 51.25 ms. The corresponding model coefficient is $\text{ME}_A/2 = -25.625$.

7.5.3 Step 3: Calculate Main Effects of B and C

By the same procedure:

$$\text{ME}_B = \frac{95+40+105+48}{4} - \frac{120+75+130+82}{4} = 72.0 - 101.75 = -29.75 \text{ ms}$$
$$\text{ME}_C = \frac{130+82+105+48}{4} - \frac{120+75+95+40}{4} = 91.25 - 82.50 = +8.75 \text{ ms}$$

More threads (B) reduce response time by 29.75 ms. Compression (C) actually *increases* response time by 8.75 ms (the CPU cost of compression outweighs the bandwidth savings).

7.5.4 Step 4: Calculate the AB Interaction

The AB column is the element-wise product of the A and B columns. We compute:

$\bar{y}_{AB,\text{concordant}} = (120+40+130+48)/4 = 84.50$ (runs where A and B have the same sign)

$\bar{y}_{AB,\text{discordant}} = (75+95+82+105)/4 = 89.25$ (runs where A and B have opposite signs)

$$\text{IE}_{AB} = 84.50 - 89.25 = -4.75 \text{ ms}$$

This is a small interaction. The effect of cache does not depend strongly on the thread pool size.

7.5.5 Step 5: Pareto Ranking

Ranking by absolute effect magnitude:

Factor	|Effect|	Cumulative %
A (Cache)	51.25	54.1%
B (Threads)	29.75	85.5%
C (Compression)	8.75	94.7%
AB	4.75	99.7%

Cache and thread pool size together account for 85.5% of the total effect—these are the "vital few." Compression and the AB interaction are minor.

7.5.6 Step 6: Confidence Intervals

Suppose from replicated runs we estimated the pooled standard deviation as $s = 6.0$ ms. With $N = 8$ runs, the standard error of any main effect is:

$$\text{SE} = \frac{2s}{\sqrt{N}} = \frac{2 \times 6.0}{\sqrt{8}} = 4.24 \text{ ms}$$

For a 95% confidence interval with, say, $\nu = 8$ degrees of freedom ($t_{0.025,8} = 2.306$):

Factor	Effect	95% CI
A (Cache)	-51.25	$[-61.03, -41.47]$
B (Threads)	-29.75	$[-39.53, -19.97]$
C (Compression)	$+8.75$	$[-1.03, +18.53]$

7.5.7 Step 7: Conclusions

1. **Cache (A)** is the dominant factor. Its CI is entirely negative (reduces response time). Enable the cache.
2. **Thread pool (B)** is the second most important factor. Its CI is entirely negative. Use 16 threads.
3. **Compression (C)** is not statistically significant—its CI includes zero. The 8.75 ms increase could be noise. Leave compression off (it also saves CPU).
4. The **AB interaction** is small and not significant. Cache and threads help independently.

5. **Predicted best configuration**: Cache on, 16 threads, compression off. Predicted response: $86.875 - 25.625 - 14.875 - 4.375 = 42.0$ ms. (This matches run 4 closely.)

7.6 Statistical Significance

7.6.1 Using Confidence Intervals

For each main effect, compute the 95% confidence interval:

$$\text{CI} = \text{ME} \pm t_{0.025,\nu} \cdot \text{SE}$$

If the interval does not contain zero, the effect is statistically significant. Report the confidence interval alongside the point estimate—it conveys both the magnitude and the uncertainty.

7.6.2 Practical vs. Statistical Significance

A factor can be statistically significant (the CI excludes zero) but practically unimportant (the effect is tiny). Conversely, a factor can be practically important but not statistically significant (the experiment was too small to detect it). Always consider both.

Do Not Over-Rely on p-Values

A p-value tells you how unlikely the observed data would be if the true effect were zero. It does *not* tell you the probability that the effect is real, nor the magnitude of the effect. Focus on effect sizes and confidence intervals.

Chapter 8

Response Surface Methodology

8.1 From Effects to Models

Screening and factorial designs answer the question: "Which factors matter?" Response Surface Methodology (RSM) answers: "What are the optimal settings?"

RSM fits a polynomial model to the data and uses the model to predict the response at untested factor combinations, including the predicted optimum.

8.2 What Is a Response Surface?

Imagine your response is the elevation on a landscape. Each factor is a direction you can walk—north-south, east-west. A *linear model* says the landscape is a flat tilted plane: walking north always goes uphill at the same rate, no matter where you are. A *quadratic model* allows hills and valleys: there can be a peak (maximum) somewhere in the middle, and walking past it starts going downhill.

Response Surface Methodology is about mapping this landscape with a minimum number of measurements, then finding the highest peak (if you want to maximize) or the lowest valley (if you want to minimize). Instead of measuring the elevation at thousands of grid points, you strategically place a few dozen measurement stations (your experimental runs) and fit a smooth mathematical surface through them. The surface tells you not just what the response is *at* your measurement points, but predicts what it would

be *between* them—including the location of the peak.

The power of RSM is that a quadratic model with k factors requires only about $2k^2$ measurements to fit, while a fine grid search would require hundreds or thousands. With 3 factors and 20 well-placed runs, you can map a landscape that a $10 \times 10 \times 10 = 1000$-point grid would struggle to reveal.

8.3 Fitting the Model

Given n observations, the linear model in matrix form is:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

The least-squares estimate is:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

For a linear model with k factors, $\mathbf{X}$ is an $n \times (k+1)$ matrix (including the intercept column). For a quadratic model, $\mathbf{X}$ has $1 + 2k + \binom{k}{2}$ columns (intercept + linear + squared + interaction terms).

8.4 Model Adequacy

8.4.1 R^2 and Adjusted R^2

R^2 always increases when you add terms to the model, even if those terms are pure noise. Adjusted R^2 penalizes for model complexity:

- $R^2 > 0.9$: Excellent fit
- $0.7 < R^2 < 0.9$: Good fit, some unexplained variation
- $R^2 < 0.7$: Consider adding terms or transforming variables

A large gap between R^2 and R^2_{adj} suggests overfitting.

8.4.2 Residual Analysis

Always check the residuals $e_i = y_i - \hat{y}_i$:

- Plot residuals vs. fitted values: should show no pattern
- Plot residuals vs. run order: should show no trend (drift)
- Normal probability plot of residuals: should be approximately linear

8.5 Finding the Optimum

For a quadratic model, the stationary point (where the partial derivatives are zero) can be found analytically:

$$\hat{\mathbf{x}}_s = -\frac{1}{2}\mathbf{B}^{-1}\mathbf{b}$$

where $\mathbf{b}$ is the vector of linear coefficients and $\mathbf{B}$ is the matrix of quadratic and interaction coefficients.

If the stationary point is a maximum (all eigenvalues of $\mathbf{B}$ are negative), it is the predicted optimum for a "maximize" response. If it is a minimum (all eigenvalues positive), it is the optimum for "minimize." If eigenvalues have mixed signs, the stationary point is a saddle point, and the optimum lies on the boundary of the design space.

Confirmation Runs

Never trust a predicted optimum without verification. Run 3–5 confirmation experiments at the predicted settings. If the observed response matches the prediction (within the confidence interval), the model is validated.

8.6 When the Optimum Is Outside Your Design Space

Sometimes the stationary point computed from your quadratic model falls outside the region you actually tested. For example, you tested temperature from 150 to 200°C, but the model predicts the optimum at 220°C. This does *not* mean you should blindly set temperature to 220°C—the model is a polynomial approximation that may be wildly inaccurate outside the tested range.

Instead, use the **method of steepest ascent** (or steepest descent, for minimization):

1. From your current factorial or RSM analysis, identify the direction of greatest improvement. This is determined by the linear coefficients of your model: the factor with the largest positive coefficient (for maximization) points in the most promising direction.
2. Starting from the center of your current design, take a series of steps along this direction. Each step increases (or decreases) each factor

in proportion to its coefficient. For example, if the temperature coefficient is twice the pressure coefficient, increase temperature by 10 units for every 5 units of pressure increase.

3. Run a few experiments (3–5) along this path, spaced out enough to see the trend. The response will initially improve, then plateau or decline.
4. Center your next RSM design at the point where the response was best (just before it started declining). Run a new CCD or Box–Behnken there to find the precise optimum in that neighborhood.

Steepest Ascent Is Iterative

Think of steepest ascent as hiking uphill in fog. You cannot see the summit, but you can feel which direction is steepest. You walk that way until the ground levels off, then stop and survey the local area more carefully (a new RSM design). You may need several rounds of "hike, then survey" to reach the true summit.

8.7 Multi-Response Optimization

When multiple responses must be optimized simultaneously (e.g., maximize yield while minimizing cost), the *desirability function* approach is standard:

1. For each response i, define a desirability d_i that maps the response to a $[0, 1]$ scale (0 = unacceptable, 1 = ideal).
2. Compute the overall desirability as the geometric mean: $D = (d_1 \cdot d_2 \cdot \ldots \cdot d_m)^{1/m}$
3. Find the factor settings that maximize D.

Part IV

Practice

Chapter 9

Planning Your First Experiment

Before you choose a design type, select factor levels, or write a configuration file, you need to answer several fundamental questions. Skipping this planning step is the single most common reason experiments fail or produce useless results. This chapter provides a systematic checklist and a worked example to guide you through the process.

9.1 The Pre-Experiment Checklist

Before You Design Anything, Answer These Questions

Work through this checklist in order. Do not proceed to design selection until you have clear answers for every item.

9.1.1 1. What Am I Trying to Learn or Optimize?

State your objective in one sentence. Be specific. Not "make the system faster" but "maximize throughput in requests per second under realistic load." Not "understand the process" but "identify which 3 of these 12 factors have the largest effect on yield."

Your objective determines everything else:

- **Screening** ("which factors matter?") → Plackett–Burman or fractional factorial.
- **Characterization** ("how do they interact?") → full factorial.

- **Optimization** ("what are the best settings?") → CCD or Box–Behnken.

9.1.2 2. What Factors Can I Control?

List *all* candidate factors, even ones you are unsure about. It is better to screen a factor and discover it does not matter than to omit it and miss something important. For each factor, note:

- Is it continuous (temperature, pressure) or categorical (algorithm A vs. B)?
- Can I set it precisely and reproducibly?
- Are there any constraints (safety limits, hardware limits)?

9.1.3 3. What Are Reasonable Low and High Levels?

Do Not Pick Levels Too Close Together

If your low and high levels are too similar, the effect will be smaller than the noise and you will conclude (incorrectly) that the factor does not matter. A good rule of thumb: the difference between levels should span a range you believe could produce a noticeable change. If you are unsure, err on the side of a wider range—you can always narrow it later.

For each factor, choose levels that:

- Are practically achievable (do not set temperature to 5000°C)
- Span a range where you expect the response to change
- Avoid known failure regions (the system crashes above 90% CPU)
- Are far enough apart to produce a detectable signal

9.1.4 4. What Will I Measure?

Define each response precisely, including:

- **Name:** What you call it (throughput, yield, latency)
- **Units:** req/sec, %, milliseconds
- **How measured:** Which tool, which metric, over what time window
- **Direction:** Are you maximizing or minimizing?

Vague responses like "performance" are a recipe for confusion. "p99 latency in milliseconds, measured by wrk2 over a 60-second steady-state window" is unambiguous and reproducible.

9.1.5 5. How Noisy Is My System?

Run 5–10 identical experiments (same settings) and measure the response. The standard deviation tells you the noise floor. This determines:

- Whether you need replication (high noise → more replicates)
- How far apart your factor levels need to be (the effect must exceed the noise)
- Whether DOE is even the right approach (if the noise is so large that no effect can be detected, fix the measurement system first)

9.1.6 6. How Many Runs Can I Afford?

Be realistic about your budget in time, money, or compute. This constrains your design choice:

- **8–12 runs:** Screening designs (PB, fractional factorial)
- **16–32 runs:** Full factorials on 4–5 factors
- **15–50 runs:** Response surface designs (CCD, Box–Behnken)

It is always better to run a smaller, well-chosen design than a larger design you cannot complete.

9.1.7 7. Are There Nuisance Variables I Should Block On?

Nuisance variables are things that affect the response but are not factors you want to study—time of day, batch of raw material, which server you run on, which operator performs the test. If you know a nuisance variable exists, use blocking:

- Assign each block a complete (or balanced) subset of the design
- The block effect is estimated and removed from the error, giving you sharper estimates of the effects you care about

9.2 Worked Example: Planning a Web Server Optimization

Let us walk through the checklist for a concrete scenario.

Planning Worksheet: Nginx Optimization

1. Objective: Maximize requests per second (throughput) for a static file workload served by Nginx, while keeping p99 latency below 50 ms.

2. Candidate factors:

Factor	Type	Low	High
`worker_processes`	continuous	2	16
`worker_connections`	continuous	256	2048
`sendfile`	categorical	off	on
`tcp_nopush`	categorical	off	on
`keepalive_timeout`	continuous	5 s	65 s
`gzip`	categorical	off	on

3. Level rationale: Worker processes range from 2 (well below the 8-core CPU) to 16 (double the cores). Worker connections span a 8× range. The categorical factors are simply on/off toggles.
4. Responses:

- Throughput: requests/sec, measured by `wrk` over 30 s steady state, maximize.
- p99 latency: milliseconds, measured by `wrk`, minimize (constraint: must stay below 50 ms).

5. Noise estimate: Five identical runs at default settings gave throughput values of 12,340, 12,510, 12,280, 12,450, and 12,390. Standard deviation ≈ 90 req/s (about 0.7% of the mean). Noise is low; single replicates may suffice for screening.
6. Budget: Each run takes about 2 minutes. We can afford up to 20 runs in a single session.
7. Blocking: We will run everything on the same machine in one session, so no blocking is needed. If we had to split across two days, we would block on day.
Decision: Six factors, budget of 20 runs. A Plackett–Burman design with 12 runs screens all six factors. That leaves 8 runs for a follow-up factorial on the top 3 factors. Total: 20 runs.

9.3 Common Pitfalls in Planning

1. **Starting with optimization before screening.** If you have more than 4–5 factors, do not jump straight to a CCD. Screen first, then optimize the vital few.

2. **Forgetting to define the response precisely.** "Performance" is not a response. "Median throughput in req/s over a 60-second window

after 10 seconds of warm-up" is a response.

3. **Choosing levels based on convenience, not signal.** Levels of 100 MB and 110 MB for a buffer size are probably too close. Levels of 100 MB and 1 GB will produce a detectable effect.

4. **Ignoring nuisance variables.** If your results depend on time of day, server load, or network conditions, and you do not block on them, they inflate your error and hide real effects.

5. **Not estimating noise before designing.** Without a noise estimate, you cannot know how many replicates you need or whether your factor levels are far enough apart. Always do a few preliminary runs.

6. **Planning more runs than you can complete.** An incomplete design is often unanalyzable. It is better to run an 8-run PB to completion than to abandon a 32-run factorial halfway through.

Chapter 10

The Sequential Experimentation Strategy

10.1 Phase 1: Screening

Goal: Identify the vital few factors from a large list of candidates.

Design: Plackett–Burman or Resolution III fractional factorial.

Runs: 8–20 (regardless of how many factors you have).

Analysis: Pareto chart, main effects. Ignore interaction estimates (they are confounded). Rank factors by absolute effect magnitude.

Decision: Select the top 3–5 factors for further study. Fix the rest at their cheapest, easiest, or most robust level.

10.2 Phase 2: Characterization

Goal: Understand main effects *and* interactions among the important factors.

Design: Full factorial (2^k with $k = 3$–5) or Resolution V fractional factorial.

Runs: 8–32.

Analysis: Full ANOVA, interaction effects, confidence intervals. Check for curvature by including center points—if the center-point mean differs significantly from the average of the corner points, curvature exists.

Decision: If no curvature, the optimal settings are at the best corner. If curvature exists, proceed to Phase 3.

10.3 Phase 3: Optimization

Goal: Find the optimal factor settings within the design space.

Design: Central Composite Design or Box–Behnken.

Runs: 15–50 (depending on the number of factors).

Analysis: Fit a quadratic RSM model. Check R^2, residuals, and the location of the stationary point.

Decision: Identify the predicted optimum. If it is within the design space, proceed to confirmation. If it is on the boundary, consider shifting the design space and running a new experiment.

10.4 Phase 4: Confirmation

Goal: Validate the predicted optimum.

Design: 3–5 replicate runs at the predicted optimal settings.

Analysis: Compare the observed mean to the predicted value. If the observed value falls within the prediction interval, the model is confirmed.

Chapter 11

Applications

11.1 Software Performance Benchmarking

Software systems have many tunable parameters—thread pool sizes, buffer capacities, timeout values, caching strategies—and engineers often tune them by intuition, one at a time. DOE provides a systematic alternative.

11.1.1 Example: Database Configuration

A database administrator suspects five configuration knobs affect transaction throughput:

Factor	Low	High
`buffer_pool_size`	1 GB	4 GB
`io_capacity`	200	2000
`max_connections`	100	500
`query_cache`	0 MB	256 MB
`thread_pool_size`	4	16

Phase 1: A Plackett–Burman design with 8 runs reveals that `buffer_pool_size` and `io_capacity` account for 75% of the throughput variation. The other three knobs have negligible effects.

Phase 2: A 2^2 full factorial on the two important knobs (4 runs, with 2 center points) confirms a significant interaction: large buffer pools only help when I/O capacity is also high.

Phase 3: A CCD with 13 runs on the two knobs finds the optimal settings: `buffer_pool_size` = 3.2 GB, `io_capacity` = 1600.

Total: 27 runs. An OVAT approach would have tested each knob at 3–4 values (≈15–20 runs) and missed the interaction entirely.

11.1.2 Example: Compiler Flags

A performance engineer wants to understand how compiler optimization level (`-O1`, `-O2`, `-O3`), link-time optimization (`off`, `thin`, `full`), and architecture target (`native`, `x86-64-v3`) affect runtime and binary size.

A $3 \times 3 \times 2 = 18$ full factorial tests all combinations. The analysis reveals that `-O3` with `thin` LTO produces the best runtime, but `full` LTO produces the smallest binary. The interaction between optimization level and LTO is significant: `full` LTO improves `-O1` more than `-O3` (diminishing returns when the optimizer has already done most of the work).

11.2 Scientific and Engineering Experiments

11.2.1 Example: Chemical Reactor Optimization

A chemical engineer wants to maximize yield and purity while minimizing cost in a batch reactor. Three factors are controlled: temperature (150–200°C), pressure (2–6 bar), and catalyst concentration (0.5–2.0 g/L).

Running the reactor at all extremes simultaneously (200°C, 6 bar, 2.0 g/L) risks a runaway reaction. A Box–Behnken design avoids this: each run has at most two factors at their extremes while the third is at center.

With 15 runs, the engineer fits a quadratic model for each response and discovers that:

- Temperature has the largest effect on yield (quadratic, with a maximum near 185°C).
- Catalyst concentration has the largest effect on purity (linear, higher is better).
- Cost increases linearly with all three factors, with a temperature–pressure interaction.

The desirability function approach finds the compromise: 185°C, 3.5 bar, 1.8 g/L, yielding 78% yield, 95% purity, at $55/batch.

11.2.2 Example: Materials Science

A materials scientist studies the tensile strength of a new alloy as a function of four processing parameters. A CCD with 31 runs reveals that annealing temperature has a strong quadratic effect (optimum at 520°C) and interacts

with cooling rate. The quadratic model achieves $R^2 = 0.94$ and correctly predicts the confirmation runs within $\pm 3\%$.

11.2.3 Example: Microservice Configuration

A platform team needs to optimize a Kubernetes-deployed microservice for latency and cost. The controllable factors are:

Factor	Low	High
`replicas`	2	8
`cpu_limit`	500m	2 cores
`memory_limit`	512 Mi	2 Gi
`hpa_target_cpu`	50%	80%

A $2^4 = 16$ full factorial gives the complete picture. The analysis reveals that more replicas only help up to a point (interaction with CPU limit), and that doubling memory has no effect when CPU is the bottleneck. The team sets `replicas=4`, `cpu_limit=1.5`, `memory_limit=512Mi`, `hpa_target_cpu=60%`—saving 40% on compute cost with no latency increase.

11.3 Machine Learning Hyperparameter Tuning

DOE can replace random search or grid search for hyperparameter optimization. The efficiency gains are dramatic:

Approach	Runs (8 params)	What You Learn
Grid search (3^8)	6,561	Everything, wastefully
Random search	$\sim$100	Noisy, unstructured
PB screening	12	Which 3 params matter
+ full factorial	8 more	Their interactions
Total DOE	**20**	**Focused, actionable**

- **Screening:** Use a PB design to test 10+ hyperparameters and identify the 3–4 that actually affect validation accuracy.
- **Optimization:** Use a CCD or LHS on the important hyperparameters to find the optimal region.
- **Advantage over grid search:** A 2^{10} grid search requires 1024 training runs. A PB screen requires 12.

- **Advantage over random search:** LHS guarantees better space coverage than pure random sampling with the same number of points.

Hyperparameter Screening in Practice

A data scientist tests 8 hyperparameters (learning rate, batch size, dropout, hidden layers, hidden units, optimizer, weight decay, activation) using a PB design with 12 training runs. The Pareto chart immediately shows that `learning_rate`, `batch_size`, and `hidden_units` account for 85% of the validation accuracy variation. The other 5 parameters are fixed at defaults, and a 2^3 factorial (8 runs) on the important three reveals that `learning_rate` and `batch_size` interact strongly. Total: 20 training runs instead of 6,561.

Chapter 12

Common Mistakes and How to Avoid Them

Even experienced experimenters fall into these traps. Each one can invalidate your results or waste your budget. Learn them once, avoid them forever.

12.1 Choosing Factor Levels Too Close Together

The mistake: Testing temperature at 198°C and 202°C. The true effect is tiny relative to experimental noise, so you conclude (incorrectly) that temperature does not matter.

The fix: Make the difference between levels large enough to produce a detectable effect. A rule of thumb: the difference should be at least 2–3 times the expected noise level. If your measurement varies by $\pm 5\%$, a 1% change in the factor level will be invisible.

12.2 Not Randomizing Run Order

The mistake: Running all "high temperature" experiments in the morning and all "low temperature" experiments in the afternoon. Equipment drift, ambient temperature changes, or operator fatigue get confounded with your factor.

The fix: Always randomize. Use the `--seed` flag for reproducibility. Never run experiments in the order they appear in the design matrix.

12.3 Changing Other Things Between Runs

The mistake: Updating the OS halfway through the experiment, or a colleague restarting the server. Now some runs operated under different conditions, and you cannot tell whether observed differences are due to your factors or the environment change.

The fix: Keep everything constant except the factors you are studying. If something does change unavoidably, use blocking to account for it—assign a new block at the change point.

12.4 Testing Too Many Factors Without Screening

The mistake: Running a full factorial on 8 factors (256 runs) when only 3 of them matter. You burn 253 runs on factors that contribute nothing.

The fix: Start with a Plackett–Burman screening design (12 runs for up to 11 factors). Identify the vital few, then focus your runs on those.

12.5 Ignoring Interactions

The mistake: You find that "thread count = 32" is best based on its main effect. But in reality it is only best when combined with a large connection pool. At a small connection pool, 32 threads actually makes things worse (thread contention).

The fix: Use a design that estimates interactions (full factorial or fractional factorial with Resolution IV+). Always check the interaction effects table, not just main effects.

12.6 Not Running Confirmation Experiments

The mistake: Trusting the RSM model's predicted optimum without verifying it. The model is a polynomial approximation—it may be wrong, especially if you are near the edge of the design space.

The fix: Always run 3–5 experiments at the predicted optimal settings. If the observed response matches the prediction (within the confidence interval), you are done. If not, the model needs refinement.

12.7 Confusing Replication with Repetition

The mistake: Running the benchmark 5 times without restarting the server and computing a standard deviation. This measures only *measurement* variability, not the full experimental variability that includes setup, configuration application, and environmental factors.

The fix: True replication means independently resetting the entire experimental setup between runs—restart the server, re-apply the configuration, let it warm up, then measure. Use `block_count > 1` to get genuine replicates.

	Replication	Repetition
What	Re-do entire setup	Re-measure same setup
Captures	Full experimental error	Measurement noise only
Example	Restart server, re-run	Run benchmark 3×
Use for	Estimating real variability	Estimating measurement noise

12.8 Over-Engineering the First Experiment

The mistake: Spending weeks designing a 50-run CCD as your very first experiment on a system you have never studied before.

The fix: Follow the sequential strategy. Your first experiment should be a cheap screening design (8–12 runs). Learn which factors matter, then invest runs wisely. You can always add more runs later; you cannot un-waste runs you already spent.

Chapter 13

Using the DOE Helper Tool

13.1 Installation

```
# Clone the repository
git clone <repository-url>
cd design_of_experiments

# Install dependencies
pip install -r requirements.txt

# Or install as a package
pip install -e .
```

13.2 Configuration File

The tool is driven by a JSON configuration file:

```
{
  "metadata": {
    "name": "My Experiment",
    "description": "What I am investigating"
  },
  "factors": [
    {"name": "temperature", "levels": ["150", "200"],
     "type": "continuous", "unit": "C"},
    {"name": "catalyst", "levels": ["A", "B"],
     "type": "categorical"}
```

```
    ],
    "responses": [
      {"name": "yield", "optimize": "maximize", "unit": "%"},
      {"name": "cost",  "optimize": "minimize", "unit": "USD"}
    ],
    "settings": {
      "operation": "full_factorial",
      "test_script": "my_test.sh",
      "out_directory": "results",
      "block_count": 1
    }
  }
```

13.3 Configuration Reference

This section documents every field in the JSON configuration file.

13.3.1 Metadata Fields

Field	Required	Description
`name`	Yes	A short name for the experiment. Used in report titles and file names.
`description`	No	A longer description of the experiment's purpose. Appears in the HTML report header.

13.3.2 Factor Fields

Each entry in the `factors` array describes one controllable input variable.

Field	Required	Description
`name`	Yes	Identifier for the factor. Used as the column header and command-line argument name. Must be a valid identifier (letters, digits, underscores).
`levels`	Yes	Array of level values (as strings). For 2-level designs, provide exactly 2 values. For multi-level designs, provide all values.
`type`	No	`"continuous"` (default) or `"categorical"`. Continuous factors are coded to $[-1, +1]$ for analysis. Categorical factors are compared by level means.
`unit`	No	Unit of measurement (e.g., `"MB"`, `"ms"`, `"C"`). Displayed in reports and plots.
`description`	No	Human-readable description. Shown in the HTML report.

13.3.3 Response Fields

Each entry in the `responses` array describes one measurable output.

Field	Required	Description
`name`	Yes	Identifier for the response. Must match a key in the JSON output of your test script.
`optimize`	No	`"maximize"` (default) or `"minimize"`. Determines the direction of optimization in RSM analysis.
`unit`	No	Unit of measurement (e.g., `"req/s"`, `"%"`). Displayed in reports.
`description`	No	Human-readable description of what is measured.

13.3.4 Runner Fields

The `runner` section controls how factor values are passed to your test script.

Field	Required	Description
`arg_style`	No	How factor values are passed to the test script. Options below. Default: `"double-dash"`.

The three argument styles:

`double-dash` Factors are passed as `--factor_name value`. Example: `./test.sh --temperature 200 --pressure 6`. This is the most common and explicit style.

`env` Factors are set as environment variables before invoking the script. Example: `temperature=200 pressure=6 ./test.sh`. Useful when your test script reads from the environment.

`positional` Factors are passed as positional arguments in the order they appear in the `factors` array. Example: `./test.sh 200 6`. Compact but fragile—reordering factors in the config will change which value goes where.

13.3.5 Settings Fields

Field	Required	Description
`operation`	Yes	Design type. One of: `full_factorial`, `fractional_factorial`, `plackett_burman`, `latin_hypercube`, `central_composite`, `box_behnken`.
`test_script`	Yes	Path to the executable that runs one experiment. Receives factor values and writes a JSON result file.
`block_count`	No	Number of complete replicates of the entire design. Default: 1 (no replication). Set to 2 or 3 for noisy systems.
`out_directory`	No	Directory where raw result JSON files are written. Default: `"results"`.
`processed_directory`	No	Directory where analysis outputs (plots, CSV, reports) are saved. Default: `"processed"`.
`lhs_samples`	No	Number of sample points for Latin Hypercube designs. Only used when `operation` is `latin_hypercube`. Default: 20.

13.3.6 Fixed Factors

The optional `fixed_factors` object contains key-value pairs that are passed to every run of the test script but are not varied in the design. Use these for parameters that must be set but are not part of the experiment:

```
"fixed_factors": {
  "workload": "oltp_read_write",
  "duration": "60",
  "warmup": "10"
}
```

Fixed factors appear in the runner script as additional arguments (using the same `arg_style` as the experimental factors) and are recorded in the report for reproducibility.

13.4 Supported Operations

Operation	Design Type	Requirements
`full_factorial`	All combinations	Any levels
`fractional_factorial`	2^{k-p} fraction	2 levels
`plackett_burman`	PB screening	2 levels
`latin_hypercube`	Space-filling	Any
`central_composite`	CCD	2 numeric levels
`box_behnken`	Box–Behnken	2 numeric, $\geq$3 factors

13.5 Workflow

```
# 1. Preview the design
doe info --config config.json

# 2. Generate runner script
doe generate --config config.json --output run.sh --seed 42

# 3. Execute experiments
bash run.sh

# 4. Analyze results
doe analyze --config config.json

# 5. Get optimization recommendations
doe optimize --config config.json

# 6. Generate HTML report
doe report --config config.json --output report.html

# 7. Export to CSV
doe analyze --config config.json --csv results/csv
```

13.6 Test Script Protocol

Your test script must:

1. Accept factor values via the configured argument style (`--name value`, environment variables, or positional arguments).

2. Accept `--out <path>` to specify the output file.
3. Write a JSON file to the output path with keys matching your response names.

```
#!/bin/bash
# Example test script
# ... run your experiment here ...
echo '{"yield": 85.3, "cost": 12.50}' > "$OUT_PATH"
```

13.7 Analysis Output

The `analyze` command computes and displays:

- Main effects with standard errors and 95% confidence intervals
- Two-factor interaction effects (for 2-level factors)
- Summary statistics (count, mean, std, min, max) per factor per level
- Pareto chart with cumulative contribution line
- Main effects plots (grid of line plots, one per factor)

The `optimize` command additionally fits a linear RSM model and reports R^2, coefficients, predicted optimum, and factor importance ranking.

The `report` command generates a self-contained HTML file with all tables, plots (embedded as base64 images), and a collapsible design matrix.

13.8 Troubleshooting

This section covers the most common problems users encounter and how to resolve them.

13.8.1 "My Effects Are All Near Zero"

Likely causes:

- Factor levels are too close together. If you test buffer size at 100 MB and 110 MB, the 10% difference may be invisible against the noise. Widen your levels.
- The response is dominated by noise. If the run-to-run variability is larger than any factor effect, nothing will be significant. Reduce noise (longer measurement windows, more stable environment) or increase replication.

- You are studying the wrong factors. The real drivers of performance may not be in your factor list. Revisit your candidate list and consider factors you dismissed.

13.8.2 "R^2 Is Very Low"

An R^2 below 0.5 means the model explains less than half the variation. **Likely causes:**

- Important factors are missing from the model. There may be a factor you did not include that has a large effect.
- The wrong model type is fitted. A linear model cannot capture curvature. If the true relationship is quadratic, try a response surface model.
- The system is inherently very noisy, and single replicates are insufficient. Add replication to bring down the error variance.

13.8.3 "Confidence Intervals Are Huge"

Wide confidence intervals mean the effect estimates are imprecise. **Likely causes:**

- Not enough replication. With a single replicate of a 2^k design, the error variance is estimated from higher-order interactions (which may themselves be real effects). Adding even one replicate (`block_count = 2`) dramatically improves precision.
- High inherent noise. Reduce variability by controlling nuisance variables, using blocking, or increasing measurement duration.

13.8.4 "The Predicted Optimum Is Unrealistic"

If the RSM model predicts an optimum at factor settings that are physically impossible or far outside your tested range:

- The model is extrapolating beyond the design space. Polynomial models can produce absurd predictions outside the region where data exists. Never trust predictions more than 10–20% beyond your tested range.
- The stationary point is a saddle point, not a true optimum. Check the eigenvalues. If they have mixed signs, the optimum is on the boundary of the design space, not at the stationary point.
- Consider shifting your design space toward the predicted optimum using steepest ascent (see Section on "When the Optimum Is Outside Your Design Space") and running a new experiment there.

13.8.5 "Runner Script Fails on Some Runs"

If some experimental runs succeed and others fail:

- Check the test script's error handling. Does it handle edge cases at extreme factor combinations? A setting that works at (low, low) might crash at (high, high).
- Look for missing result files. The analysis tool expects a JSON result file for every run. Missing files produce incomplete data that can bias the analysis.
- Check disk space and permissions in the output directory.
- Verify that the test script correctly parses all argument styles. A common bug: the script reads `--factor_name` but the config uses `env` style.
- If only a few runs failed, you may still be able to analyze the partial results, but the design will no longer be balanced. The effect estimates will have unequal precision, and some interactions may not be estimable.

Appendix A

Mathematical Reference

A.1 Effect Estimation Formulas

For a 2^k factorial with $N = 2^k$ runs, coded levels $x \in \{-1, +1\}$, and response vector $\mathbf{y}$:

Main effect of factor i:

$$\mathrm{ME}_i = \frac{2}{N} \sum_{j=1}^{N} x_{ij} y_j = \bar{y}_{i+} - \bar{y}_{i-}$$

Interaction effect of factors i and k:

$$\mathrm{IE}_{ik} = \frac{2}{N} \sum_{j=1}^{N} x_{ij} x_{kj} y_j$$

Standard error of any effect:

$$\mathrm{SE} = \frac{2s}{\sqrt{N}}$$

where $s^2 = \mathrm{MS}_E$ (mean square error from ANOVA or from replicates).

Sum of squares for any effect:

$$\mathrm{SS}_{\text{effect}} = \frac{N}{4} \cdot \mathrm{ME}^2$$

A.2 Response Surface Model

For k factors, the full quadratic model has $p = 1 + 2k + \binom{k}{2}$ parameters:

$$\hat{y} = b_0 + \sum_{i=1}^{k} b_i x_i + \sum_{i=1}^{k} b_{ii} x_i^2 + \sum_{i<j} b_{ij} x_i x_j$$

k	p (parameters)
2	6
3	10
4	15
5	21

The design must have at least p distinct runs (and preferably $\geq 1.5p$) to fit the model.

A.3 Minimum Run Counts by Design

	Number of Factors					
Design	2	3	4	5	6	7
Full factorial	4	8	16	32	64	128
Frac. factorial (Res. III)	–	4	8	8	8	8
Plackett–Burman	4	4	8	8	8	8
CCD	13	20	31	52	–	–
Box–Behnken	–	15	27	46	–	–

Appendix B

Glossary

ANOVA Analysis of Variance. Partitions total variation into components.

Blocking Grouping runs to account for known nuisance variation.

CCD Central Composite Design. A response surface design with factorial, star, and center points.

Coded variable A factor rescaled so that -1 = low and $+1$ = high.

Confounding When two effects cannot be distinguished because they produce identical patterns in the design matrix.

DOE Design of Experiments.

Effect The change in the mean response caused by changing a factor from one level to another.

Factor A controllable input variable in an experiment.

Fractional factorial A subset of a full factorial that trades higher-order information for fewer runs.

Interaction When the effect of one factor depends on the level of another.

Level A specific value or setting of a factor.

LHS Latin Hypercube Sampling. A space-filling design.

Main effect The average effect of a factor across all levels of other factors.

OVAT One Variable At a Time. An inefficient experimental strategy.

Pareto chart A bar chart ranking effects by magnitude.

Plackett–Burman An efficient screening design for identifying important main effects.

Randomization Randomly ordering experimental runs to protect against lurking variables.

Replication Independently repeating experimental runs.

Resolution A measure of the confounding severity in a fractional factorial.

Response A measurable outcome of an experiment.

RSM Response Surface Methodology. Fitting polynomial models to find optimal settings.

R^2 Coefficient of determination. Proportion of variance explained by the model.

References

[1] Box, G.E.P., Hunter, J.S., & Hunter, W.G. (2005). *Statistics for Experimenters: Design, Innovation, and Discovery*, 2nd ed. Wiley.

[2] Montgomery, D.C. (2017). *Design and Analysis of Experiments*, 9th ed. Wiley.

[3] Myers, R.H., Montgomery, D.C., & Anderson-Cook, C.M. (2016). *Response Surface Methodology: Process and Product Optimization Using Designed Experiments*, 4th ed. Wiley.

[4] Box, G.E.P. & Behnken, D.W. (1960). Some new three level designs for the study of quantitative variables. *Technometrics*, 2(4), 455–475.

[5] Plackett, R.L. & Burman, J.P. (1946). The design of optimum multifactorial experiments. *Biometrika*, 33(4), 305–325.

[6] Fisher, R.A. (1935). *The Design of Experiments*. Oliver & Boyd.

[7] NIST/SEMATECH e-Handbook of Statistical Methods. Available at: https://www.itl.nist.gov/div898/handbook/